



Web Controllers

Controllers

Controllers need to provide extensibility, much like **Model**, but can't use the same mechanism as the pre-requisites (a database with loaded modules) may not be available yet (e.g. no database created, or no database selected).

Controllers thus provide their own extension mechanism, separate from that of models:

Controllers are created by **inheriting** from **Controller**. Routes are defined through methods decorated with **route()**:

```
class MyController(odoo.http.Controller):
    @route('/some_url', auth='public')
    def handler(self):
        return stuff()
```

To *override* a controller, **inherit** from its class and override relevant methods, re-exposing them if necessary:

```
class Extension(MyController):
    @route()
    def handler(self):
        do_before()
        return super(Extension, self).handler()
```

- decorating with **route()** is necessary to keep the method (and route) visible: if the method is redefined without decorating, it will be “unpublished”
- the decorators of all methods are combined, if the overriding method's decorator has no argument all previous ones will be kept, any provided argument will override previously defined ones e.g.:

```
class Restrict(MyController):
    @route(auth='user')
```

will change `/some_url` from public authentication to user (requiring a log-in)

API

Routing

@ `odoo.http.route` (*route=None*, ***routing*) [\[source\]](#)

Decorate a controller method in order to route incoming requests matching the given URL and options to the decorated method.

Warning

It is mandatory to re-decorate any method that is overridden in controller extensions but the arguments can be omitted. See [Controller](#) for more details.

Parameters:

route (*Union[[str](#), Iterable[[str](#)]]*) – The paths that the decorated method is serving. Incoming HTTP request paths matching this route will be routed to this decorated method. See [werkzeug routing documentation](#) for the format of route expressions.

type (*[str](#)*) – The type of request, either `'jsonrpc'` or `'http'`. It describes where to find the request parameters and how to serialize the response.

auth (*[str](#)*) –

The authentication method, one of the following:

`'user'` : The user must be authenticated and the current request will be executed using the rights of the user.

`'bearer'` : The user is authenticated using an "Authorization" request header, using the Bearer scheme with an API token. The request will be executed with the permissions of the corresponding user. If the header is missing, the request must belong to an authentication session, as for the "user" authentication method.

`'public'` : The user may or may not be authenticated. If he isn't, the current request will be executed using the shared Public user.

`'none'` : The method is always active, even if there is no database. Mainly used by the framework and authentication modules. The request code will not have any facilities to access the current user.

methods (*Iterable[[str](#)]*) – A list of http methods (verbs) this route applies to. If not specified, all methods are allowed.

cors (*[str](#)*) – The Access-Control-Allow-Origin cors directive value.

cursor on a read-only replica instead of (by default) the primary read/write database.

handle_params_access_error (*Callable[[[Exception](#)], Response]*) – Implement a custom behavior if an error occurred when retrieving the record from the URL parameters (access error or missing error).

captcha (*str*) – The action name of the captcha. When set the request will be validated against a captcha implementation. Upon failing these requests will return a `UserError`.

save_session (*bool*) – Whether it should set a `session_id` cookie on the http response and save dirty session on disk. `False` by default for `auth='bearer'`. `True` by default otherwise.

Request

The request object is automatically set on `odoo.http.request` at the start of the request.

class `odoo.http.Request` (*httprequest*) [\[source\]](#)

Wrapper around the incoming HTTP request with deserialized request parameters, session utilities and request dispatching logic.

update_env(*user=None, context=None, su=None*) [\[source\]](#)

Update the environment of the current request.

Parameters:

user (int or `res.users record`) – optional user/user id to change the current user

context (*dict*) – optional context dictionary to change the current context

su (*bool*) – optional boolean to change the superuser mode

update_context(***overrides*) [\[source\]](#)

Override the environment context of the current request with the values of `overrides`. To replace the entire context, please use `update_env()` instead.

csrf_token(*time_limit=None*) [\[source\]](#)

Generates and returns a CSRF token for the current session

Parameters:

time_limit (*Optional[int]*) – the CSRF token should only be valid for the specified duration (in second), by default 48h, `None` for the token to be valid as long as the current user's session is.

Returns:

ASCII token string

Return type:

Is the given csrf token valid ?

Parameters:

csrf ([str](#)) – The token to validate.

Returns:

True when valid, False when not.

Return type:

[bool](#)

default_lang() [\[source\]](#)

Returns default user language according to request specification

Returns:

Preferred language if specified or 'en_US'

Return type:

[str](#)

get_http_params() [\[source\]](#)

Extract key=value pairs from the query string and the forms present in the body (both application/x-www-form-urlencoded and multipart/form-data).

Returns:

The merged key-value pairs.

Return type:

[dict](#)

make_response(data, headers=None, cookies=None, status=200) [\[source\]](#)

Helper for non-HTML responses, or HTML responses with custom response headers or cookies.

While handlers can just return the HTML markup of a page they want to send as a string if non-HTML data is returned they need to create a complete response object, or the returned data will not be correctly interpreted by the clients.

Parameters:

data ([str](#)) – response body

status ([int](#)) – http status code

a response object.

Return type:

[Response](#)

`make_json_response(data, headers=None, cookies=None, status=200)` [\[source\]](#)

Helper for JSON responses, it json-serializes `data` and sets the Content-Type header accordingly if none is provided.

Parameters:

data – the data that will be json-serialized into the response body

status ([int](#)) – http status code

headers ([List](#)[([str](#), [str](#))])) – HTTP headers to set on the response

cookies ([collections.abc.Mapping](#)) – cookies to set on the client

Return type:

[Response](#)

`not_found(description=None)` [\[source\]](#)

Shortcut for a [HTTP 404](#) (Not Found) response

`render(template, qcontext=None, lazy=True, **kw)` [\[source\]](#)

Lazy render of a QWeb template.

The actual rendering of the given template will occur at then end of the dispatching. Meanwhile, the template and/or qcontext can be altered or even replaced by a static response.

Parameters:

template ([str](#)) – template to render

qcontext ([dict](#)) – Rendering context to use

lazy ([bool](#)) – whether the template rendering should be deferred until the last possible moment

kw ([dict](#)) – forwarded to werkzeug's Response object

`reroute(path, query_string=None)` [\[source\]](#)

Rewrite the current request URL using the new path and query string. This act as a light redirection, it does not return a 3xx responses to the browser but still change the current URL.

dispatch(endpoint, args) [\[source\]](#)**JSON-RPC 2** over HTTP.

Our implementation differs from the specification on two points:

1. The `method` member of the JSON-RPC request payload is ignored as the HTTP path is already used to route the request to the controller.
2. We only support parameter structures by-name, i.e. the `params` member of the JSON-RPC request payload MUST be a JSON Object and not a JSON Array.

In addition, it is possible to pass a context that replaces the session context via a special `context` argument that is removed prior to calling the endpoint.

Successful request:

```
--> {"jsonrpc": "2.0", "method": "call", "params": {"arg1": "val1" }, "id": null}
<-- {"jsonrpc": "2.0", "result": { "res1": "val1" }, "id": null}
```

Request producing a error:

```
--> {"jsonrpc": "2.0", "method": "call", "params": {"arg1": "val1" }, "id": null}
<-- {"jsonrpc": "2.0", "error": {"code": 1, "message": "End user error message."}}
```

handle_error(exc: [Exception](#)) → [collections.abc.Callable](#) [\[source\]](#)

Handle any exception that occurred while dispatching a request to a `type='jsonrpc'` route. Also handle exceptions that occurred when no route matched the request path, that no fallback page could be delivered and that the request `Content-Type` was json.

Parameters:

exc – the exception that occurred.

Returns:

a WSGI application

dispatch(endpoint, args) [\[source\]](#)

Perform http-related actions such as deserializing the request body and query-string and checking cors/csrf while dispatching a request to a `type='http'` route.

See `load()` method for the compatible endpoint return types.

handle_error(exc: [Exception](#)) → [collections.abc.Callable](#) [\[source\]](#)

Handle any exception that occurred while dispatching a request to a `type='http'` route. Also handle exceptions that occurred when no route matched the request path, when no fallback page could be delivered and that the request `Content-Type` was not json.

Parameters:

exc ([Exception](#)) – the exception that occurred.

Returns:

a WSGI application

Response

odoo.http.Response [\[source\]](#)

alias of `odoo.http._Response`

Get Help

[Contact Support](#)[Ask the Odoo Community](#)